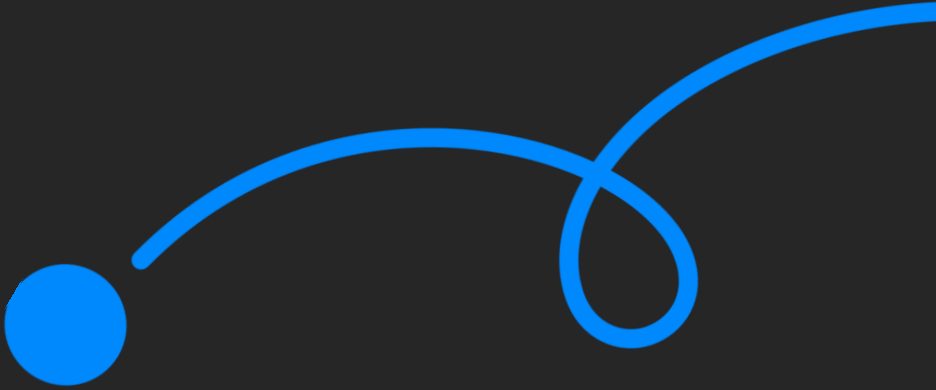




코멘토 실무PT 클래스

반도체 SW 엔지니어를 위한 새로운 SoC 위에 리눅스 디바이스 드라이버 포팅하기

멘티님, 반갑습니다!

클래스는 20:00에 시작됩니다.

출석 확인을 위해 프로필은 **실명**으로 설정해주세요.

출석을 확인합니다.



김대용

김진영

원종진

조현경

최승원

최민우

최강주

지난 세션은 어떠셨나요.

- 1 지난 주차에 어떤걸 공부했었는지 짧게 리뷰 해봐요.
- 2 복습을 진행하면서 궁금한 점이 있다면 같이 토의해봐요.
- 3 과제를 진행하면서 어려웠던 점을 알려주세요.
- 4 멘티 여러분이 작성한 과제를 다같이 살펴보도록 해요.

시작하기에 앞서

1

수업 진행을 하며
멘티님들의
이해도를 체크하며
진행할 예정입니다.

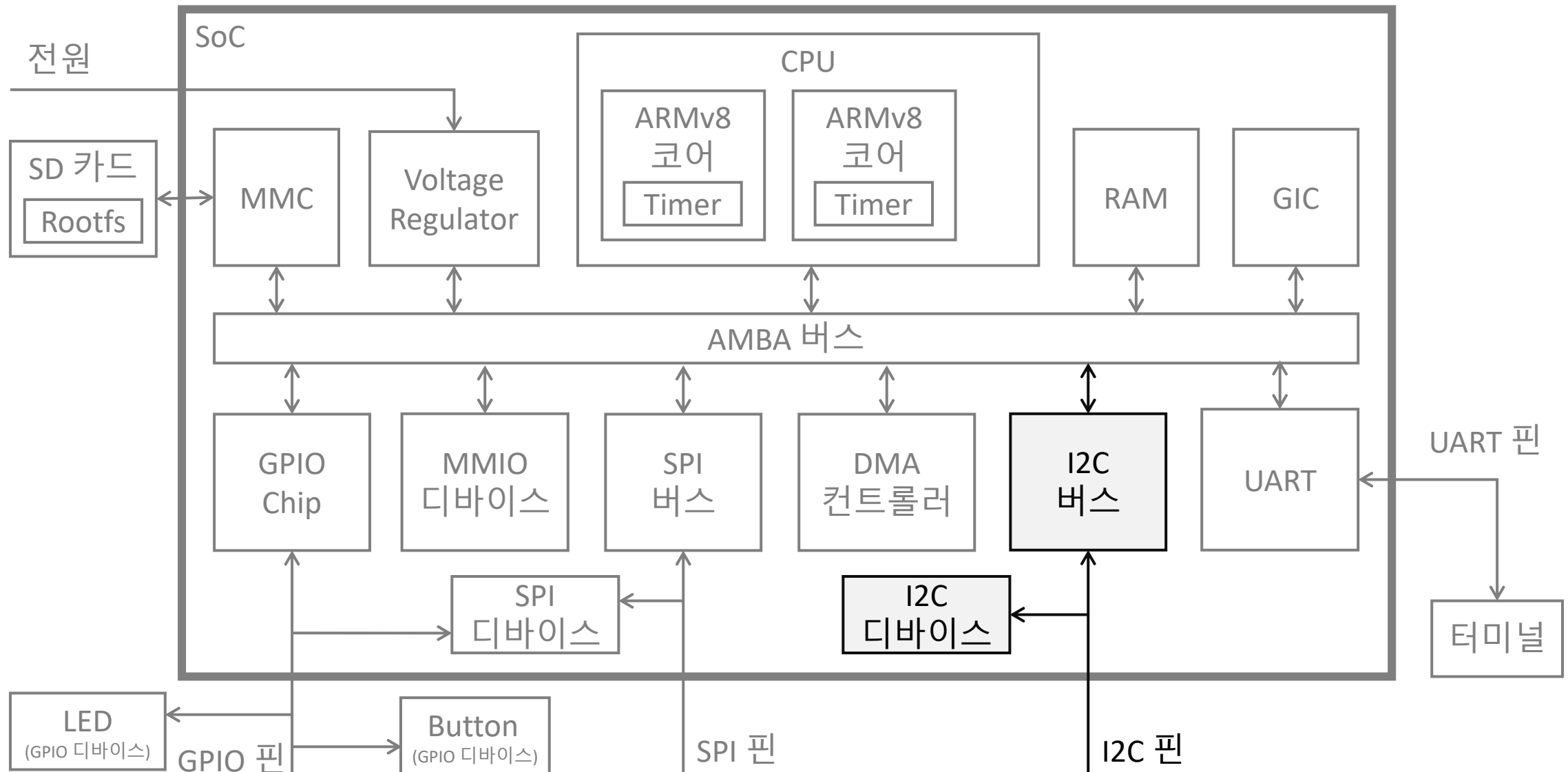
2

이해가 되지 않거나,
궁금한 내용은
채팅으로 언제든지
남겨주세요.

3

평균 50분 라이브를
진행하고, 10분의
쉬는 시간으로
진행 됩니다.

이번 수업에서 진행할 내용



01 I2C 통신 이해하기

02 새로운 I2C 하드웨어 만들기

03 I2C 툴로 I2C 하드웨어 테스트하기

04 I2C 하드웨어 드라이버 만들기

05 자주 사용되는 센서 파악하기

Chapter. 1

I2C 통신 이해하기

I2C (Inter IC, IIC), TWI(Two wired Interface) 란?

↳ 아이 스쿼어 씨라고 읽어야 함

SoC와 주변기기 또는 SoC 끼리를 연결하기 위한 인터페이스

- SoC 외부와의 연결을 위해 사용하나 SoC 내부에서 간단한 통신을 위해 사용되기도 함
- 마스터 / 슬레이브 방식 통신 지원

TWI라는 이름에서 알 수 있듯이 2개의 선만으로 버스 구성 가능

- SPI와 달리 슬레이브 개수가 늘어도 칩선택 선을 추가하지 않아도 됨

SPI에 비해 복잡한 프로토콜로 하드웨어 구현이 복잡

- 장치 주소(7bit), 레지스터 주소(8bit), 입력 데이터 (8bit), 출력 데이터 (8bit)로 구성
 - 출력 데이터는 없을 수 있음
- 주변장치는 자기에 맞는 주소인 경우에만 반응해야 함

I2C 구조

SoC와 주변기기의 2개의 신호 선끼리 연결 필요

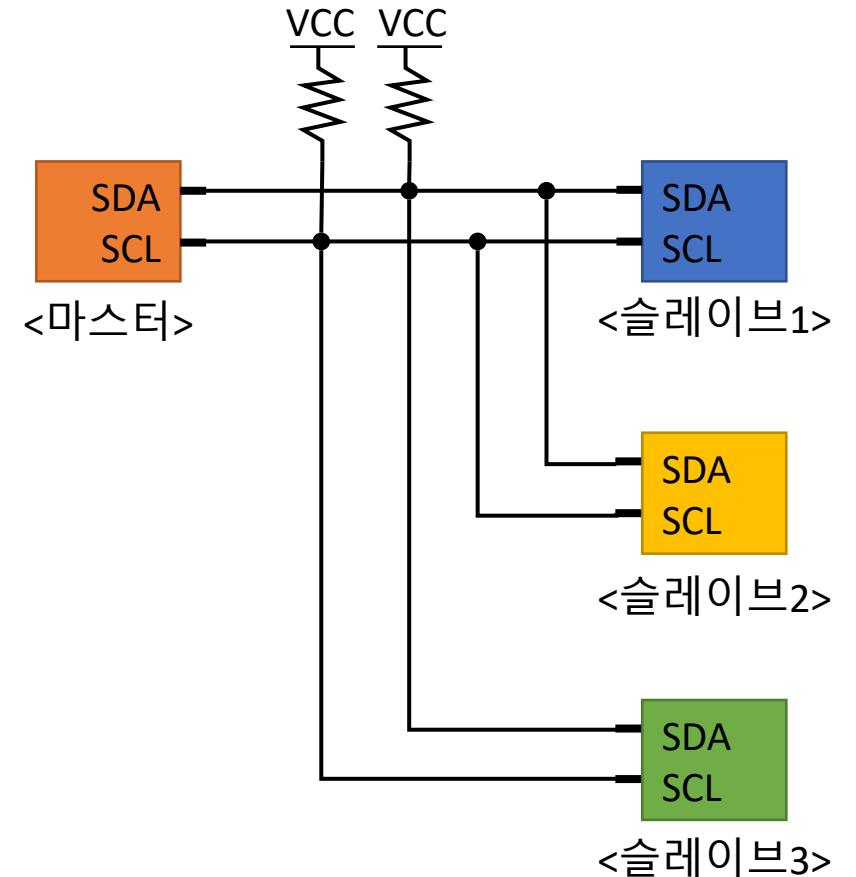
- 데이터 (SDA)
- 클럭 (SCL)

2개의 신호 선에 반드시 풀업 저항 연결 필요

장치 주소로 슬레이브를 구분하므로 칩선택 불필요

SoC와 주변기기의 VCC 전압이 꼭 같을 필요는 없음

SoC와 주변기기의 GND 끼리 연결 필요



I2C의 장점

칩선택 신호가 필요하지 않아 선을 깔끔하게 구성할 수 있음

- 각각의 주변장치가 주소로 구분됨

제대로 데이터가 전달되었는지 확인하는 ACK 비트가 존재

- 무언가의 이유로 데이터를 받지 못했을 때 파악 가능
- ACK 비트를 받지 못했다면, 데이터 다시 전송 가능

I2C의 단점

SPI에 비해서는 전송 속도가 느림

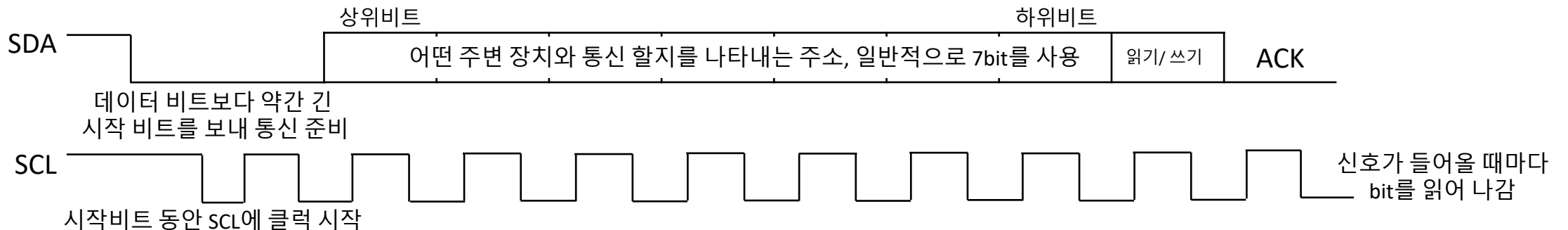
- SPI는 10Mbps 이상 전송 가능하나 SPI는 100kbps~3.4Mbps까지만 가능
- 100kbps의 경우 UART 보다도 속도가 느림
 - UART는 동시에 송/수신이 가능하며, 각각 115kbps 까지 가능
 - I2C는 동시에 송/수신이 불가능하며, 장치 주소를 보내기 위한 추가적인 시간 소요

복잡한 프로토콜 구현을 위해 하드웨어 구현 역시 복잡

- 하드웨어를 제어하기 위한 디바이스 레지스터 사용법도 SPI에 비해 복잡

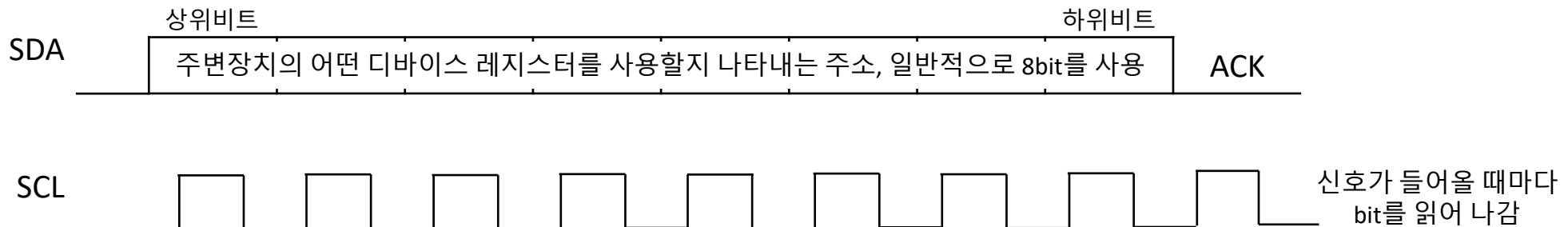
I2C 통신 흐름 - 통신의 시작

1. 통신을 시도하고자 하는 마스터가 시작 비트를 보냄
 - 다른 마스터는 자신의 동작을 멈춤, 슬레이브는 수신 준비를 함
 - 아주 동시에 마스터 여러 개가 동작하는 경우
 - 통신을 진행하다가 자신이 원하는 대로 SDA 설정이 안 되는 마스터는 전송 포기
2. 마스터는 시작비트 동안 클럭을 준비하여 SCL로 보내기 시작
3. 마스터는 클럭에 맞춰 장치 주소 7bit를 SDA로 1bit씩 보냄
4. 현재 통신이 값을 쓰는 용도라면 1을 읽기만 하는 용도라면 0으로 된 1bit를 보냄
5. 해당하는 슬레이브가 ACK 신호를 보냄



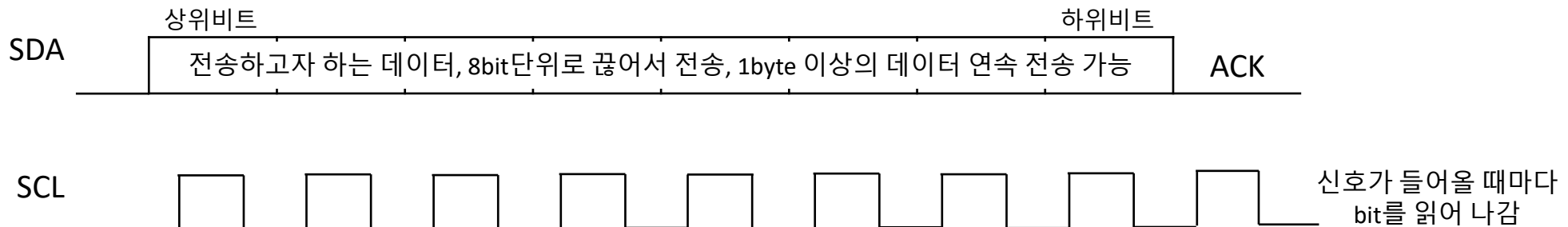
I2C 통신 흐름 - 레지스터 주소 전송

1. 마스터는 클럭에 맞춰 레지스터 주소를 SDA로 1bit씩 보냄
 - I2C는 디바이스 레지스터의 값을 읽거나 쓴다는 형태로 통신을 수행
 - 레지스터 주소는 1바이트 이상이 될 수 있음
 - 일반적으로 1바이트나 2바이트 사용
2. 8bit 전송마다 슬레이브에서 ACK 신호를 보냄
 - 무언가의 이유로 슬레이브가 ACK 신호를 보내지 못하면 풀업 저항에 의해 High
 - ACK 신호가 High인 경우, 전송이 실패했음을 의미



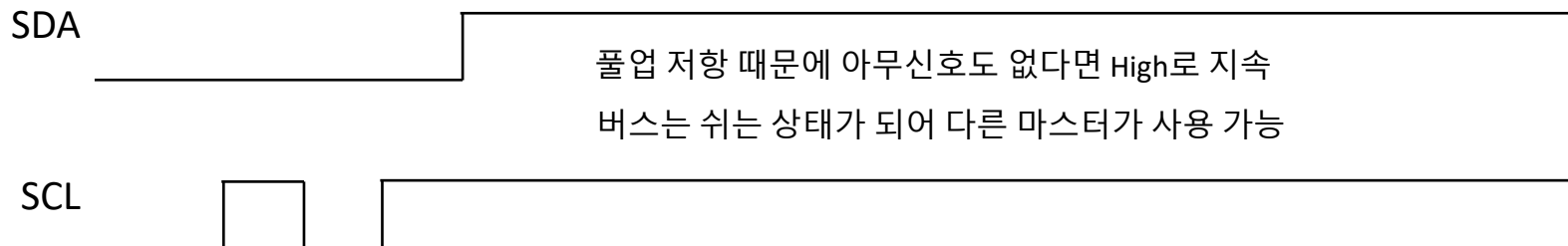
I2C 통신 흐름 - 데이터 전송

- 쓰기라면, 마스터는 클럭에 맞춰 데이터들을 SDA로 1bit씩 보냄
읽기라면, 슬레이브는 클럭에 맞춰 데이터들을 SDA로 1bit씩 보냄
 - 전송되는 데이터는 1바이트 이상이 될 수 있음
 - 일반적으로 1바이트나 2바이트 사용
- 8bit 전송마다 데이터를 받는 측에서 ACK 신호를 보냄
 - ACK 신호가 High인 경우, 전송이 실패했음을 의미



I2C 통신 흐름 - 통신의 끝

1. 데이터를 전부 송/수신했다면, 마스터는 멈춤 비트를 보냄
 - 멈춤 비트를 수신하면, 해당 슬레이브도 통신 과정을 마무리
2. 마스터는 더 이상 SCL에 클럭을 보내지 않음



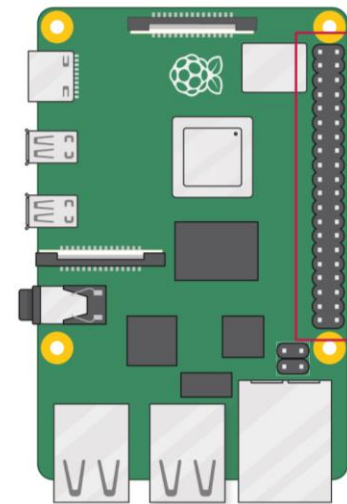
실제 SoC에서 I2C의 형태

라즈베리 파이에 사용되는 SoC(Broadcom社 BCM27xx)는 I2C 연결을 지원

- I2C를 사용하는 다양한 형태의 주변기기 (센서, 메모리, LCD 등) 연결 가능

외부로 나온 핀 헤더를 사용하여 I2C 사용 가능

- GPIO와 겸용으로 사용 되는 핀
 - I2C로 사용하도록 사전 설정 필요



3V3 power	1	2	5V power
GPIO 2 (SDA)	3	4	5V power
GPIO 3 (SCL)	5	6	Ground
GPIO 4 (GPCLK0)	7	8	GPIO 14 (TXD)
Ground	9	10	GPIO 15 (RXD)
GPIO 17	11	12	GPIO 18 (PCM_CLK)
GPIO 27	13	14	Ground
GPIO 22	15	16	GPIO 23
3V3 power	17	18	GPIO 24
GPIO 10 (MOSI)	19	20	Ground
GPIO 9 (MISO)	21	22	GPIO 25
GPIO 11 (SCLK)	23	24	GPIO 8 (CE0)
Ground	25	26	GPIO 7 (CE1)
GPIO 0 (ID_SD)	27	28	GPIO 1 (ID_SC)
GPIO 5	29	30	Ground
GPIO 6	31	32	GPIO 12 (PWM0)
GPIO 13 (PWM1)	33	34	Ground
GPIO 19 (PCM_FS)	35	36	GPIO 16
GPIO 26	37	38	GPIO 20 (PCM_DIN)
Ground	39	40	GPIO 21 (PCM_DOUT)

※ 출처 : 라즈베리 파이 문서

I2C를 사용하는 주변기기 예시

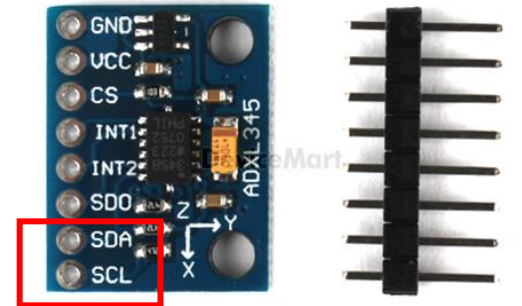
많은 배선 없이 간단하게 붙일 수 있는 주변 장치

- 간단한 값을 출력하는 각종 센서
 - 예시 : <https://www.devicemart.co.kr/goods/view?no=10918253>



많은 센서들이 I2C와 SPI 통신 둘다 지원

- 가속도 센서, 온습도 센서 등
 - 예시 : <https://www.devicemart.co.kr/goods/view?no=1314305> 많은 주변장치들이 I2C와 SPI 통신 겸용



높은 프레임을 요구하지 않는 LCD 디스플레이

- 예시 : <https://www.devicemart.co.kr/goods/view?no=12718636>

간단한 데이터를 저장하기 위한 주변장치 (EEPROM 등)

- 예시 : <https://www.devicemart.co.kr/goods/view?no=1056518>





질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

Chapter. 2

새로운 I2C 하드웨어 만들기

I2C 컨트롤러란?

I2C 통신은 I2C 컨트롤러라는 별도의 하드웨어를 사용

- 개발자는 I2C 통신의 구체적인 구현을 신경 쓸 필요가 없음

시작 / 멈춤 비트, ACK 비트를 송수신하고 장치 주소를 관리하는 등 까다로운 구현

SoC 외부와 통신해야 하므로 물리적인 특성을 고려해야 함

- SCL 클럭에 맞추어 1비트씩 SDA 핀으로 데이터 송수신

QEMU의 새 SoC에 I2C 컨트롤러 추가하기

```
...
#include "hw/i2c/i2c.h"
...
struct ComentoMachineState {
    ...
    DeviceState *i2c;
};
...
// 우리가 추가하는 I2C 컨트롤러의 경우 인터럽트는 사용하지 않음
#define COMENTO_BASE_I2C        0x09016000
...
static void create_i2c(ComentoMachineState *vms, MemoryRegion *mem) {
    hwaddr base = COMENTO_BASE_I2C;
    SysBusDevice *s;
    char name[] = "i2c";

    // ARM SBCON I2C 컨트롤러 장치 추가
    // I2C bitbang, ARM versatile이라고도 불림
    vms->i2c = qdev_new("versatile_i2c");

    // QMP 스크립트에서 I2C 버스를 위해 사용할 이름 지정
    qdev_set_id(vms->i2c, name, NULL);

    s = SYS_BUS_DEVICE(vms->i2c);
    sysbus_realize_and_unref(s, &error_fatal);
    memory_region_add_subregion(mem, base, sysbus_mmio_get_region(s, 0));
}
```

```
...
static void machcomento_init(MachineState *machine)
{
    ...
    create_i2c(vms, sysmem);
}
```

<hw/arm/comento.c>

디바이스 트리와 커널 설정에 I2C 컨트롤러 추가하기

```
...
i2c@9016000 {
    compatible = "arm,versatile-i2c";
    // I2C 컨트롤러의 인터럽트와 MMIO 주소 명시
    reg = <0x00 0x9016000 0x00 0x1000>;
    // 인터럽트는 사용하지 않으므로 명시하지 않음
    // SPI와는 달리 칩선택을 명시할 필요 없음
    // SPIDEV 처럼 I2C의 주변장치를 명시할 필요도 없음

};
...
```

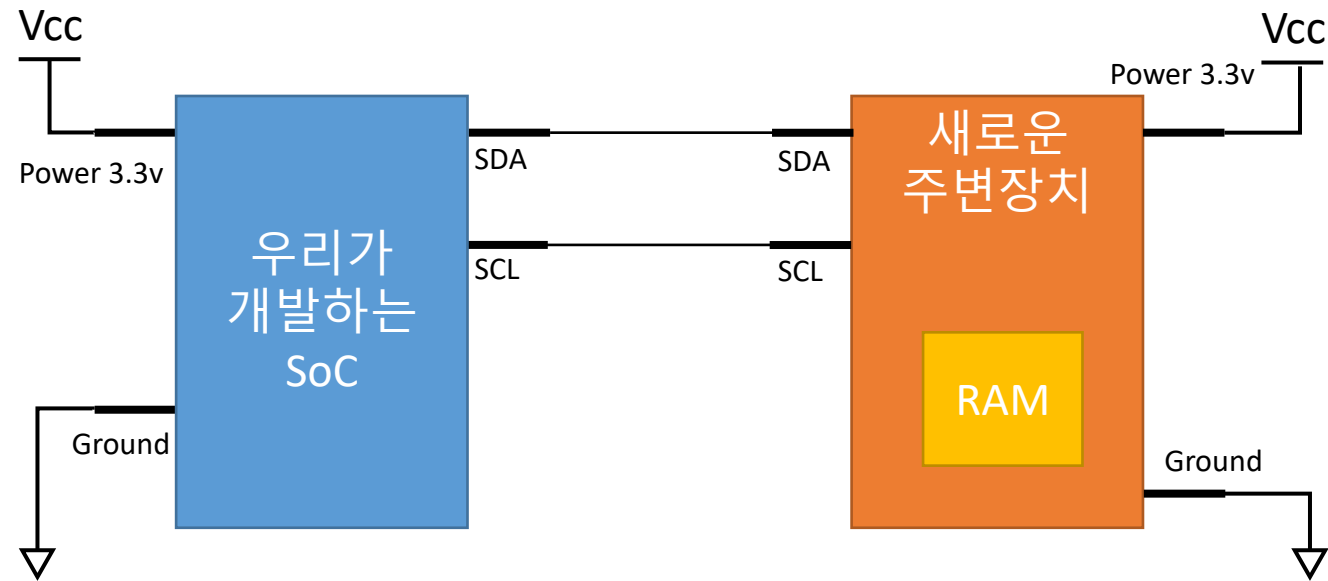
<arch/arm64/boot/dts/comento/comento.dts>

```
...
CONFIG_I2C=y
CONFIG_I2C_VERSATILE=y
# I2C 툴을 사용하기 위해 필요한 설정
# CHARDEV를 추가하지 않으면 /dev에 I2C 컨트롤러 디바이스 노드가 없음
CONFIG_I2C_CHARDEV=y
```

<arch/arm64/configs/comento_defconfig>

수정한 defconfig를 적용하기 위해
make comento_defconfig 명령어 사용

수업에서 사용할 I2C의 시나리오



➤ 위 회로와 비슷하게 I2C 사용 하드웨어를 QEMU로 에뮬레이션 해봅시다.

- 장치 주소는 0x40으로 지정
- 주변장치는 256바이트의 RAM을 가지고 있으며 I2C로 읽거나 쓸수 있음
 - 쓸 때는 레지스터 주소 1바이트와 데이터 1바이트를 마스터가 보냄
 - 읽을 때는 레지스터 주소 1바이트를 마스터가 보내고 슬레이브가 데이터 1바이트를 보냄

QEMU에 I2C를 사용하는 주변장치 추가하기 (1/2)

```
#include "qemu/osdep.h"
#include "hw/sysbus.h"
#include "hw/irq.h"
#include "hw/i2c/i2c.h"
#include "sysemu/runstate.h"
#include "qapi/visitor.h"

#define TYPE_I2C_COMENTO "i2c-comento"
#define STORAGE_SIZE 256

OBJECT_DECLARE_SIMPLE_TYPE(I2C_COMENTO_State, I2C_COMENTO)

struct I2C_COMENTO_State {
    SysBusDevice parent_obj;
    uint8_t storage[STORAGE_SIZE];
    uint8_t addr;
    int cycle;
};

// I2C 통신을 처리하기 위한 함수
// SPI와는 다르게 수신과 송신을 각각 다른 함수로 구현
// 마스터 입장에서 데이터를 받을 때 호출되는 함수
static uint8_t i2c_comento_rx(I2CSlave *obj)
{
    I2C_COMENTO_State *s = I2C_COMENTO(obj);
    // 슬레이브에서는 현재 주소의 데이터를 반환
    // 현재 주소는 마스터가 이전에 보내줬어야 함
    return s->storage[s->addr];
}

// 마스터 입장에서 데이터를 보냈을 때 호출되는 함수
static int i2c_comento_tx(I2CSlave *obj, uint8_t data)
```

```
{
    I2C_COMENTO_State *s = I2C_COMENTO(obj);
    // 슬레이브에서 2바이트(디바이스 레지스터 주소, 쓸 데이터)를
    // 수신하는 경우가 있으므로 cycle을 사용해 현재 몇 번째 데이터를
    // 받고 있는지 알아냄
    switch (s->cycle++) {
    case 0:
        s->addr = data; // 0번 사이클이면 현재 주소 갱신
        break;
    case 1:
        // 1번 사이클이면 데이터를 현재 주소에 쓰기
        s->storage[s->addr] = data;
        break;
    default:
    }
    return 0;
}

static int i2c_comento_event(I2CSlave *obj, enum i2c_event event)
{
    I2C_COMENTO_State *s = I2C_COMENTO(obj);

    switch (event) {
    // 시작 비트일 때 현재 cycle을 0으로 초기화
    case I2C_START_SEND:
        s->cycle = 0;
        break;
    default:
    }
    return 0;
}
```


QEMU에 SPI를 사용하는 하드웨어 추가하기 (2/2)

```
static void i2c_comento_class_init(ObjectClass *klass, void *data)
{
    I2CSlaveClass *k = I2C_SLAVE_CLASS(klass);

    k->event = i2c_comento_event;
    k->recv = i2c_comento_rx;
    k->send = i2c_comento_tx;
}

static const TypeInfo i2c_comento_info = {
    .name          = TYPE_I2C_COMENTO,
    .parent        = TYPE_I2C_SLAVE,
    .instance_size = sizeof(I2C_COMENTO_State),
    .class_init    = i2c_comento_class_init,
};

static void i2c_comento_register_types(void)
{
    type_register_static(&i2c_comento_info);
}

type_init(i2c_comento_register_types)
```

<hw/misc/comento/i2c.c>

```
...
softmmu_ss.add(when: 'CONFIG_COMENTO', if_true: files('i2c.c'))
```

<hw/misc/comento/meson.build>

```
static void create_i2c_devs(ComentoMachineState *vms) {
    I2CBus *i2c = (I2CBus *)qdev_get_child_bus(vms->i2c, "i2c");
    // I2C 버스에 슬레이브 주변장치를 추가하면서 장치 주소는 0x40으로 설정
    i2c_slave_create_simple(i2c, "i2c-comento", 0x40);
}

...

static void machcomento_init(MachineState *machine)
{
    ...
    create_i2c_devs(vms);
}
```

<hw/arm/comento.c>

1. hw/misc/comento/에 I2C를 사용하는 "i2c-comento" 하드웨어 추가
2. hw/arm/comento.c에 해당 하드웨어를 추가하고 I2C 컨트롤러의 I2C 버스와 연결

Rootfs에 I2C 툴 추가하기

I2C 버스에 연결된 주변장치를 사용하기 위해서는 2가지 방법

- I2C 컨트롤러의 디바이스 노드 (SPIDEV와 달리 실제 제품 사용 가능)를 사용하는 I2C툴 사용
- 주변 장치 전용의 디바이스 드라이버를 구현 → 본 수업에서는 다루지 않습니다.

I2C 툴은 I2C 컨트롤러의 디바이스 노드 사용을 위해 만들어진 명령어 도구

- 리눅스에서 관리 : <https://git.kernel.org/pub/scm/utils/i2c-tools/i2c-tools.git/about/>

Rootfs에 I2C 툴을 추가할 수 있도록 빌드루트 설정

- make menuconfig
 - Target packages -> Show packages that are also provided by busybox : 선택
 - Target packages -> Hardware handling -> i2c-tools : 선택

빌드 루트 빌드하기

- make -j<코어 개수>

Roofs를 SD 카드 이미지로 복사하기 (1/2)

1. 이미지 파일을 Loop 디바이스로 설정

- 이미지 파일을 바로 마운트 불가, Loop 디바이스 상태에서 마운트 가능
- `sudo losetup -Pf --show sdcard.img`

```
mentoring@comento:~$ sudo losetup -Pf --show sdcard.img  
/dev/loop8
```

- 출력되는 디바이스 노드 파일이 설정된 loop 디바이스

2. Loop 디바이스의 첫 번째 파티션을 ext4 형식으로 포맷

- `sudo mkfs.ext4 <loop 디바이스 경로>p1`

```
mentoring@comento:~$ sudo mkfs.ext4 /dev/loop8p1  
mke2fs 1.46.5 (30-Dec-2021)  
Discarding device blocks: done  
Creating filesystem with 16128 4k blocks and 16128 inodes  
  
Allocating group tables: done  
Writing inode tables: done  
Creating journal (1024 blocks): done  
Writing superblocks and filesystem accounting information: done
```

Rootfs를 SD 카드 이미지로 복사하기 (2/2)

3. 포맷한 파티션과 빌드루트에서 생성된 이미지 동시에 마운트

- `mkdir mnt1 mnt2`
- `sudo mount -o loop <loop 디바이스 경로>p1 mnt1`
- `sudo mount -o loop <빌드루트 디렉토리>/output/images/rootfs.ext4 mnt2`

4. 마운트된 빌드루트 이미지의 내용을 마운트된 포맷 파티션으로 복사

- `sudo cp -R mnt2/* mnt1/.`

5. 마운트한 디렉토리를 모두 언마운트

- `sync; sudo umount mnt1 mnt2`

6. Loop 디바이스 해제

- `sudo losetup -d <loop 디바이스 경로>`



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 3

I2C 툴로 I2C 하드웨어 테스트하기

I2C 툴 사용하여 I2C 테스트 하기

I2C 컨트롤러 디바이스 노드를 사용하여 I2C 통신 수행

- i2cdetect : I2C 컨트롤러를 탐지하고 I2C 버스 내의 주변장치 탐지
- -l : I2C 컨트롤러 디바이스 노드의 목록을 출력
 - 디바이스 노드/dev/ 밑에 i2c-<컨트롤러 번호>의 형태
 - 탐지된 순서대로 컨트롤러 번호가 부여
- <컨트롤러 번호> : 해당 컨트롤러의 I2C 버스 내의 주변 장치 탐지
- i2cget <컨트롤러 번호> <장치 주소> <레지스터 주소>: 주변 장치에서 데이터를 읽음
- i2cset <컨트롤러 번호> <장치 주소> <레지스터 주소> <데이터>: 데이터를 씀
- i2cdump <컨트롤러 번호> : 주변장치의 모든 데이터를 출력

I2C 툴 사용하여 I2C 탐지 실습

```
# i2cdetect -l
i2c-0 i2c Versatile I2C adapter I2C adapter
# i2cdetect 0
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will probe file /dev/i2c-0.
I will probe address range 0x08-0x77.
Continue? [Y/n] Y
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- --
40: 40 -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- -- -- -- --
# ls -lah /dev/i2c-*
crw----- 1 root  root    89,  0 Jan  1 00:00 /dev/i2c-0
#
```

i2cdetect -l로 I2C 컨트롤러 디바이스 노드 탐지

i2cdetect <컨트롤러 번호>로 버스에 붙어있는 주변장치 탐지

- 앞서 QEMU에서 추가했던 장치 주소가 0x40인 디바이스가 탐지됨

I2C 툴 사용하여 I2C 테스트 실습 (1/2)

```
# i2cset 0 0x40 0x1 0x1
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will write to device file /dev/i2c-0, chip address 0x40,
data address 0x01, data 0x01, mode byte.
Continue? [Y/n] y
# i2cset 0 0x40 0x2 0x1
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will write to device file /dev/i2c-0, chip address 0x40,
data address 0x02, data 0x01, mode byte.
Continue? [Y/n] Y
# i2cget 0 0x40 0x0
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will read from device file /dev/i2c-0, chip address 0x40, data address
0x00, using read byte data.
Continue? [Y/n] y
0x00
# i2cget 0 0x40 0x1
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will read from device file /dev/i2c-0, chip address 0x40, data address
0x01, using read byte data.
Continue? [Y/n] Y
0x01

# i2cget 0 0x40 0x2
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will read from device file /dev/i2c-0, chip address 0x40, data address
0x02, using read byte data.
Continue? [Y/n] Y
0x01
# i2cset 0 0x40 0x1 0xff
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will write to device file /dev/i2c-0, chip address 0x40,
data address 0x01, data 0xff, mode byte.
Continue? [Y/n] Y
# i2cget 0 0x40 0x1
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will read from device file /dev/i2c-0, chip address 0x40, data address
0x01, using read byte data.
Continue? [Y/n] Y
0xff
```

I2C 주변장치와 직접 통신하여 데이터를 조작하므로
정말 수행할 것인지 확인하는 메시지가 나옴

i2cset으로 0x01 레지스터에 1, 0x02 레지스터에 1 입력

- i2cget으로 레지스터에 값이 잘 써졌는지 확인

i2cset으로 0x01 레지스터에 0xff 입력

- i2cget으로 레지스터에 값이 잘 써졌는지 확인

I2C 툴 사용하여 I2C 테스트 실습 (2/2)

```
# i2cdump 0 0x40
No size specified (using byte-data access)
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will probe file /dev/i2c-0, address 0x40, mode byte
Continue? [Y/n] Y
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 00 ff 01 00 00 00 00 00 00 00 00 00 00 00 00 00  ..?.....
10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
```

i2cdump로 앞서 i2cset으로 입력했던 내용이 있음을 확인

Chapter. 4

I2C 사용자 프로그램 만들기

I2C를 사용하는 프로그램 만들기

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/i2c-dev.h> // ioctl을 위해 i2c 헤더 파일을 include 함

int main() {
    char data[256];
    char buf[2];
    int fd, i;

    // /dev에 생긴 I2C 컨트롤러의 디바이스 노드 파일 경로로 열기
    fd = open("/dev/i2c-0", O_RDWR);
    if (fd < 0) {
        fprintf(stderr, "Failed to open the device file\n");
        return 1;
    }

    // 장치 주소를 먼저 ioctl로 설정
    if (ioctl(fd, I2C_SLAVE, 0x40) < 0) {
        fprintf(stderr, "Failed to ioctl\n");
        return 1;
    }

    for (i = 0; i < 256; i++) {
        buf[0] = i;
        // 레지스터 주소를 먼저 디바이스 노드에 씀
        if (write(fd, buf, 1) < 0) { // 슬레이브에게 레지스터 주소 전달
            fprintf(stderr, "Failed to write\n");
            return 1;
        }
    }
}
```

```
// 슬레이브에서 반환한 데이터를 디바이스 노드에서 읽음
if (read(fd, data + i, 1) < 0) {
    fprintf(stderr, "Failed to read\n");
    return 1;
}

printf("Previous data : %s\n", data);

printf("New data : ");
scanf("%255s", data);

for (i = 0; i < 256; i++) {
    buf[0] = i;
    buf[1] = data[i]; // 레지스터 주소와 데이터를 디바이스 노드에 씀
    if (write(fd, buf, 2) < 0) {
        fprintf(stderr, "Failed to write\n");
        return 1;
    }
}
return 0;
}
```

<i2c-sample.c>

- 빌드루트의 툴체인으로 프로그램 빌드
- <빌드루트디렉토리>/output/host/bin/aarch64-buildroot-linux-gnu-gcc i2c-sample.c -o i2c-sample

I2C를 사용하는 프로그램을 SD카드로 복사

SD 카드 이미지에 프로그램 복사

- `sudo losetup -Pf --show sdcard1.img`
- `sudo mount <loop 경로>p1 /mnt`
- `sudo cp i2c-sample /mnt/usr/bin/.`
- `sync; sudo umount /mnt`
- `sudo losetup -d <loop 경로>`

I2C를 사용하는 프로그램 실습

```
comento@comento:~$ buildroot-2023.08/output/host/bin/aarch64-buildroot-linux-gnu-gcc -o i2c-sample i2c-sample.c
comento@comento:~$ sudo losetup -Pf --show sdcard.img;
/dev/loop1
comento@comento:~$ sudo mount /dev/loop1p1 /mnt
comento@comento:~$ sudo cp i2c-sample /mnt/usr/bin/.
comento@comento:~$ sync
comento@comento:~$ sudo umount /mnt
comento@comento:~$ sudo losetup -d /dev/loop1
```

```
# i2c-sample
Previous data :
New data : HelloWorld~~~
# i2c-sample
Previous data : HelloWorld~~~
New data : I2C-Sample-comento
# i2cdump 0 0x40
No size specified (using byte-data access)
WARNING! This program can confuse your I2C bus, cause data loss and worse!
I will probe file /dev/i2c-0, address 0x40, mode byte
Continue? [Y/n] Y
    0 1 2 3 4 5 6 7 8 9 a b c d e f 0123456789abcdef
00: 49 32 43 2d 53 61 6d 70 6c 65 2d 63 6f 6d 65 6e I2C-Sample-comen
10: 74 6f 00 00 00 00 00 00 00 00 00 00 00 00 00 to.....
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
a0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
e0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
#
```

<빌드 및 SD 카드 이미지로 복사>

<실행 결과>



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.



쉬는 시간!

조금 쉬었다가, 다시 만나요!

Chapter. 5

자주 사용되는 센서 파악하기

온도 센서 – 써미스터 (Thermistor)

접촉식 온도 센서로 가장 많이 사용되고 있는 센서

- 소형 전자 온도계 부터 에어컨 실외기까지 다양한 분야에서 사용됨

온도에 따라 저항 값이 각각 다른 특성을 활용

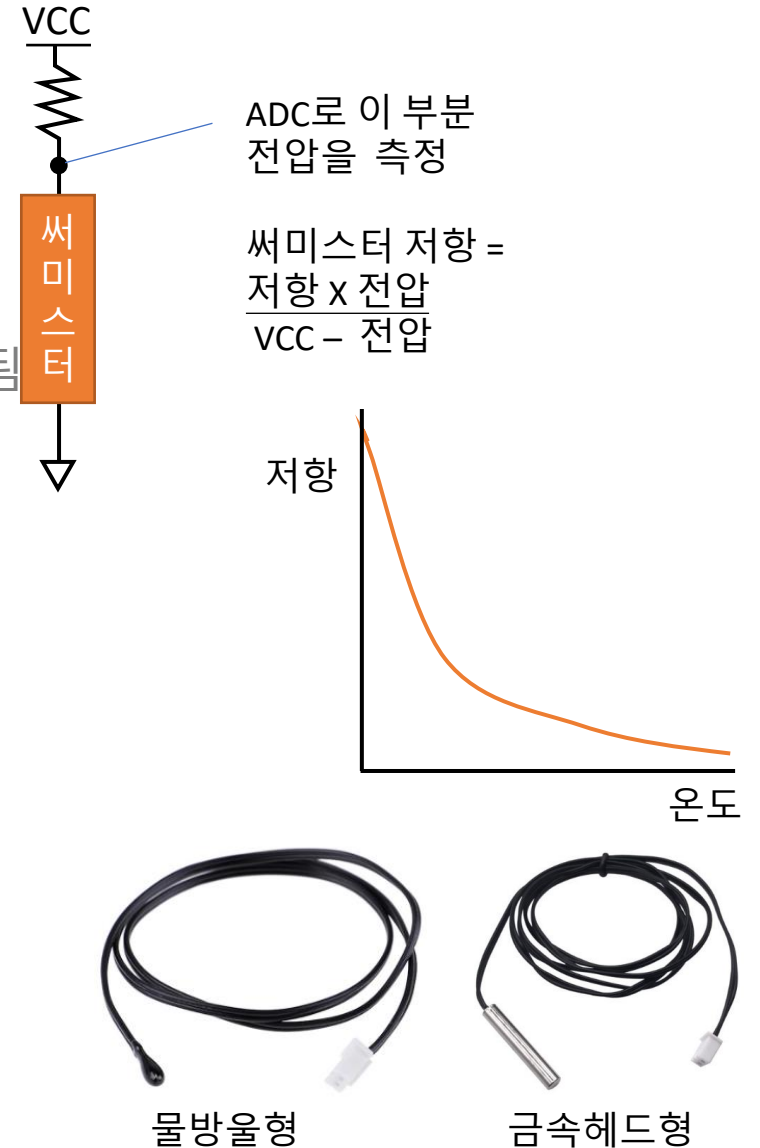
- 전압분배회로와 ADC를 사용하여 저항 값 측정 가능
- 온도가 올라갈수록 저항이 줄어듬

장점 및 단점

- 장점 : 온도 변화를 빠르게 감지할 수 있음, 가격이 매우 저렴함
- 단점 : 온도 변화에 따른 저항 변화가 비선형
 - 별도의 변환 테이블 또는 복잡한 계산 요구

예시

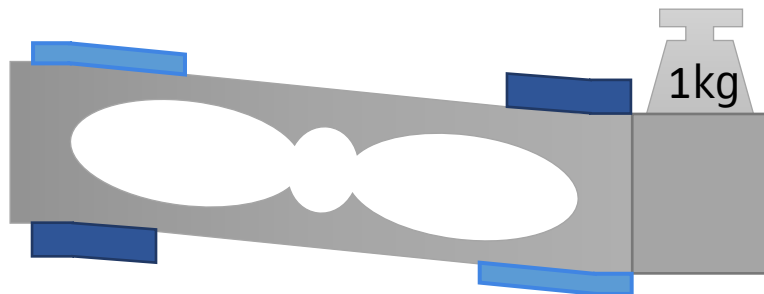
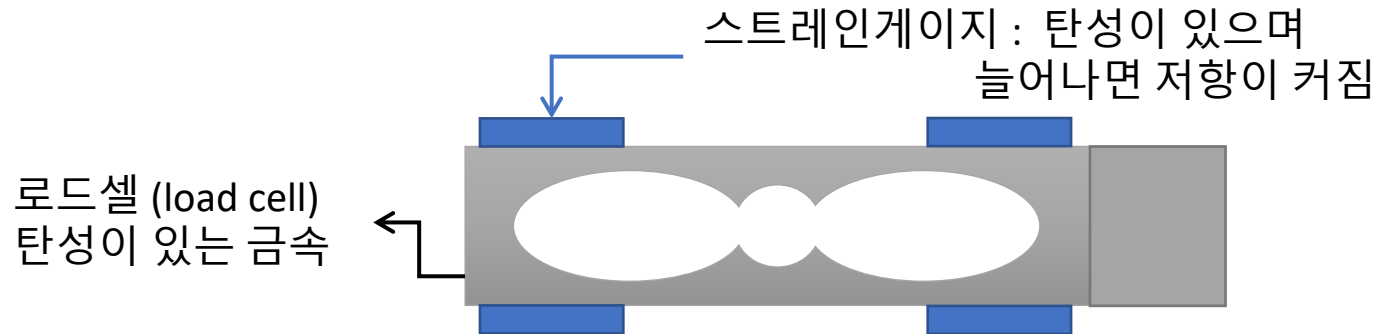
- <https://www.devicemart.co.kr/goods/view?no=1065874>



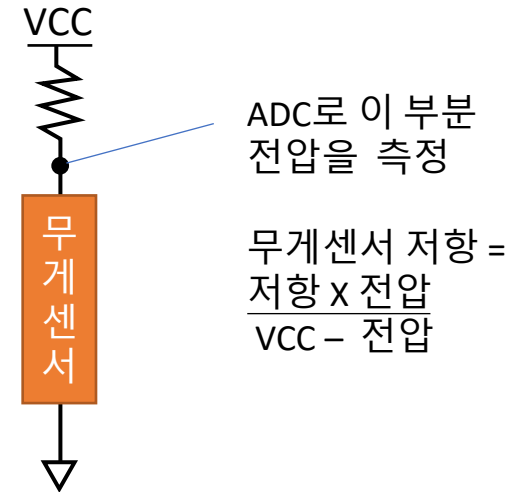
무게 센서

무게에 따라 발생하는 저항의 변화를 측정

- 전압분배회로와 ADC를 이용하여 측정



- 로드셀 가운데의 구멍으로 인해 굽혀지는 차이 발생
- 스트레인 게이지가 늘어나고 줄어들면서 저항 변화



광 센서

어떤 종류의 빛을 감지할 것인지에 따라 다양한 센서 존재

CdS 셀 센서 : 빛을 받으면 저항이 작아지고, 빛이 없으면 저항이 커짐

- 가시광선을 잘 검출하며, 매우 저렴한 가격
- 급격한 밝기 변화는 감지하지 못함 – 전반적인 밝기 정도만 검출하는 용도
- 예시 : <https://www.devicemart.co.kr/goods/view?no=11364>



포토 다이오드 / 트랜지스터 : 빛을 받으면 전류가 흐르고, 빛이 없으면 전류가 흐르지 않음

- 빠른 반응속도 – 광통신 등에 사용 가능
- 일반적으로 필터를 내장하여 적외선 센서에 많이 사용 됨 – 검정 LED 처럼 생김
- 예시 : <https://www.devicemart.co.kr/goods/view?no=1160069>

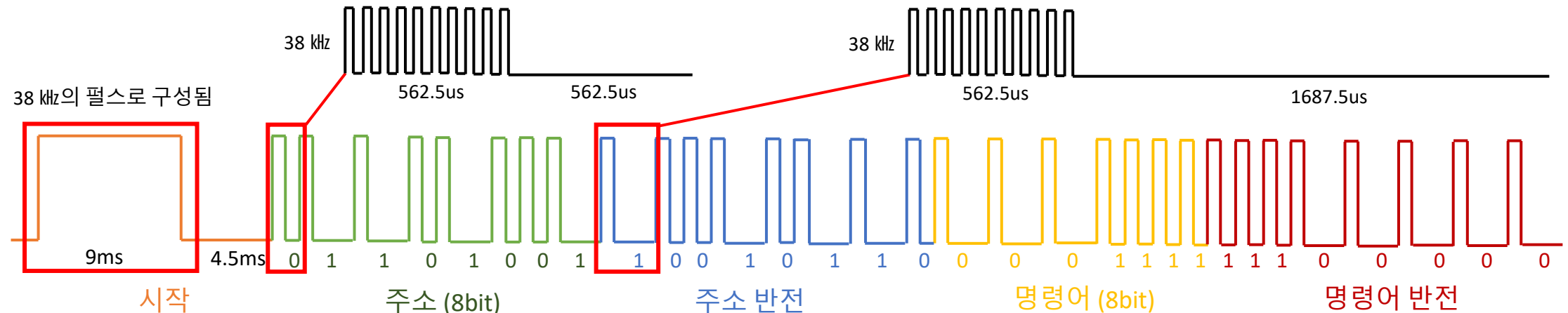


광 센서의 활용 - 리모컨

적외선을 통한 광통신, 일반적으로 NEC 표준을 사용하여 38kHz의 주파수 사용

빛의 특성상 노이즈가 생기기 쉬우므로 독특한 방식 사용

- 전기 신호처럼 High와 Low 대신, 펄스 뒤에 오는 Low 신호의 길이가 짧으면 0, 길면 1을 사용
 - 38 kHz의 독특한 주파수이므로 특정 주파수만 찾아내는 필터로 펄스를 걸러낼 수 있음
- 주소와 명령어를 각각 보내면서 반전 값도 같이 보내 노이즈에 대처하기 쉽게 함



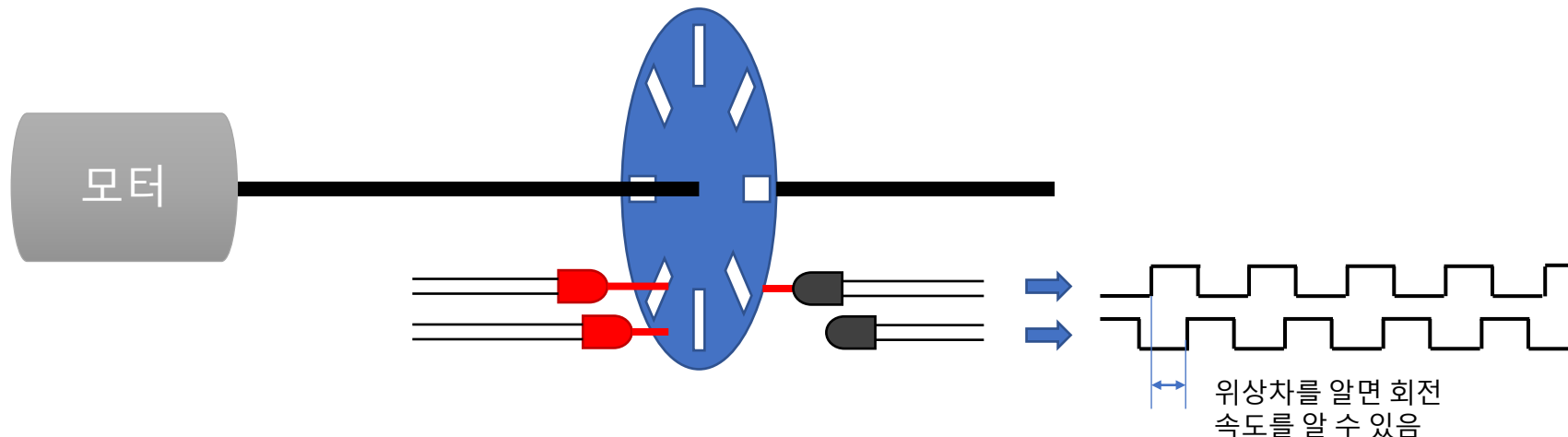
광 센서의 활용 - 엔코더

모터의 회전 속도를 알기 위한 센서

- 일반적인 DC나 AC 모터로는 부하(load)에 따라 회전속도가 달라질 수 있음
 - 같은 전력을 사용하더라도 무거운 작업을 수행하면 회전속도가 느려짐

모터와 구멍이 뚫린 원판을 연결하고 원판에 LED와 광센서 2쌍을 부착

- 생기는 펄스의 위상 차이를 통해 현재 회전속도 계산



광 센서의 활용 - 동작 / 거리 감지

눈에 안 보이는 적외선을 쏘아서 반사되는 빛의 양을 측정

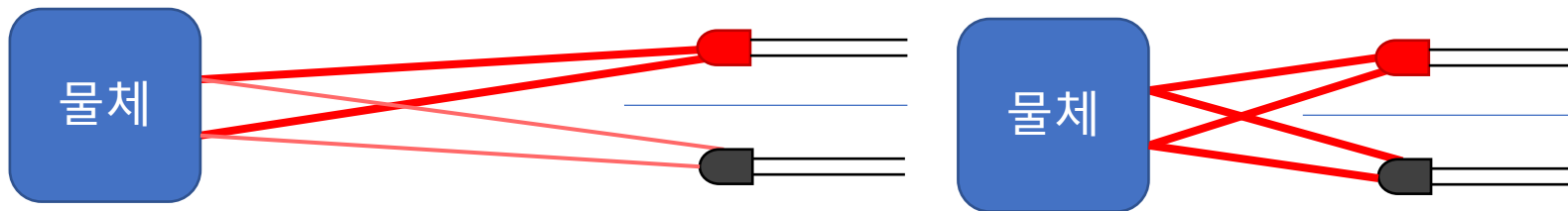
동작 센서 : 광센서로 들어오는 적외선의 양의 변화를 측정

근접 센서 : 광센서로 들어오는 적외선의 양의 크기로 거리를 측정

- 핸드폰에서 전화를 받을 때 화면을 끄는 용도로 많이 활용

거리 센서 : 적외선을 펄스의 형태로 쏘고 광센서로 들어오는 펄스의 위상차로 거리 측정

- 정밀한 거리 측정을 위해서 레이저 이용

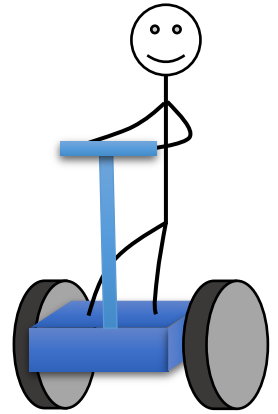


물체와의 거리에 따라
반사되는 적외선 양 변화

가속도 / 자이로 센서

가속도 센서 : 가속도를 측정하여 물체의 움직임의 변화를 측정

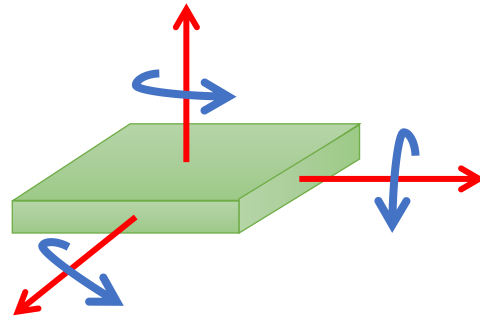
- 핸드폰 화면의 기울기 측정, 게임기, 세그웨이(Segway) 등에 활용
- 중력에 의해 기본 가속도가 항상 측정됨
 - 지면 방향으로 9.8 m/s^2 ,이를 응용하여 현재 기울기 정도 측정 가능



바퀴가 두개인 개인용 이동장치
오뚜기처럼 균형을 잡아줌

자이로 센서 : 물체의 회전 속도를 측정하여 물체의 현재 상태를 측정

- 드론 (Drone) 비행시 균형을 위해 사용



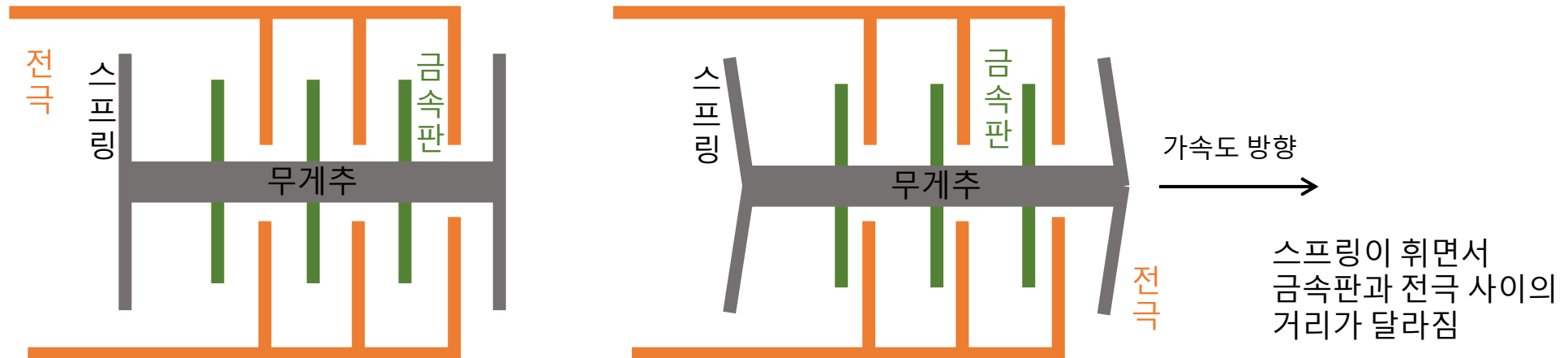
가속도 센서는 빨간색 방향의 힘을 측정
자이로 센서는 파란색 방향의 힘을 측정

사용자가 몸을 기울이면
기울인 방향으로 가야
균형이 잡을 수 있는 것을
이용하여 조종하게 됨

가속도 센서의 원리

매우 작은 크기(수백 μm)의 전극과 스프링으로 구성

- 관성에 따라 스프링에 매달린 금속이 흔들리면서 금속과 전극 사이의 거리 차이가 생김
- 금속과 전극 사이의 거리에 따라 정전 용량이 달라짐
 - 정전용량이란? 금속판을 가까이 두었을 때 전기에너지를 임시적으로 저장할 수 있는 성질
 - 현재의 정전용량을 측정하여 금속과 전극 사이의 거리를 알 수 있음
 - 금속과 전극 사이의 거리를 통해 현재 가해지는 힘의 크기와 방향을 알 수 있음



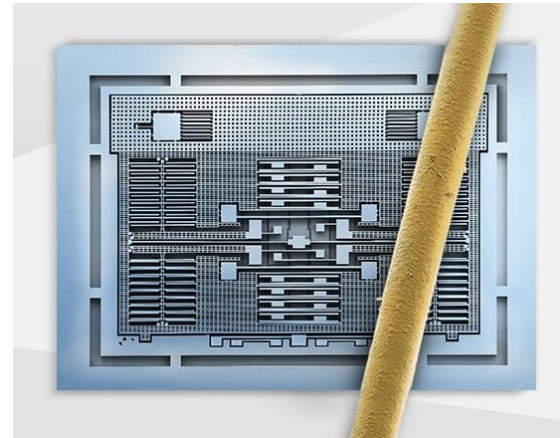
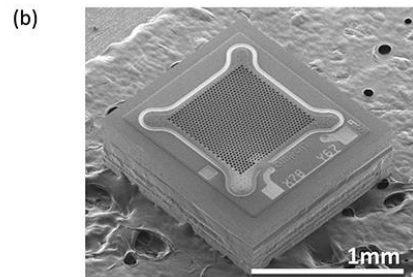
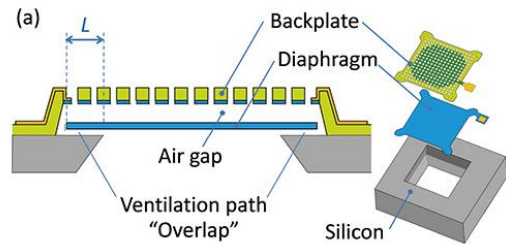
MEMS (Micro Electro Mechanical System)

μm 단위의 초소형 전자기계 시스템

- 매우 작은 크기의 기계 장치를 만드는 제조 기술의 정점

가속도 센서, 자이로 센서, 마이크 등 기계적인 센서에 활용

- 빛이나 화학물질을 검출하는 센서에서도 많이 활용되기 시작함



MEMS 기술을 활용한 가속도 센서

(노란색 막대기는 머리카락)

출처 : <https://www.bosch-mobility.com/en/solutions/electronic-components/mems-sensors/>

MEMS 기술을 활용한 초소형 마이크

출처 : <https://www.omron.com/global/en/technology/omrontechnics/vol50/012.html>



질문해주세요!

이해되지 않거나, 궁금한 내용은 채팅과 마이크를 통해 질문해주세요.
질문이 많을수록 더 많은 걸 알려드릴 수 있어요.

코멘토 실무PT 클래스

6주간 수고하셨습니다

